

RECOMMEND(order_items, boxes):

Expand quantities into individual items.

Calculate total product weight and volume.

Sort boxes by (cost, internal volume, stable ID).

For each box:

Reject if total weight exceeds capacity.

Reject if total volume exceeds internal volume.

Reject if any item cannot fit individually in any rotation.

Sort items largest-first.

Start candidate positions with (0, 0, 0).

For each item:

Try candidate positions in deterministic order.

At each position, try all distinct allowed rotations.

Accept the first placement that:

stays inside the box, and

does not overlap any placed item.

If accepted:

Record its position and rotated dimensions.

Add positions beside, behind, and above it.

Otherwise:

Mark this box as "packing not found".

Try the next box.

If every item was placed:

Return this box and its placement details.

Return "no suitable box found" with rejection reasons.

Why the preliminary checks are insufficient

Consider **two cubes measuring $2 \times 2 \times 2$ cm** and a box measuring **$3 \times 3 \times 3$ cm**, with enough weight capacity.

- Combined item volume: **16 cm^3** .
- Box volume: **27 cm^3** .
- Each cube fits individually.

However, both cubes cannot fit together. To avoid overlap, they must be separated along at least one axis. That would require **$2 + 2 = 4$ cm**, but the box provides only **3 cm** along every axis.

Weight, volume, and individual fit are therefore **necessary checks, not proof of collective fit**.

Stacking versus a basic 3D heuristic

Approach	How it works	Advantages	Limitations
Single vertical stack	Position items above one another; check footprints and combined height	Very easy to implement and explain	Misses arrangements that use space beside items
Corner-placement heuristic	Try rotations at positions beside, behind, and above placed items	Uses all three dimensions and returns an inspectable layout	Greedy choices can miss feasible arrangements

I recommend the **corner-placement heuristic** for this assignment. It demonstrates actual geometric packing while remaining understandable.

This specific heuristic is a new recommendation: we had agreed on rotations and box selection, but had not selected an algorithm.

Implementation

Download the complete service: `box_recommendation.py`.

Place it in your Django app's services package. It has no Django imports, database writes, or HTTP response handling. Its entry point is:

```
result = recommend_box(order_items, boxes)
```

Pass an iterable of `OrderItem` instances with their products loaded, and an iterable of `Box` instances. It uses the field names proposed earlier.

The implementation explicitly uses these **provisional policies**, because they remained unresolved:

- No additional padding or clearance; exact fits are allowed.
- Capacity is compared against **product weight only**.
- Geometry permits stacking, without checking support or fragility.
- Measurements come from current `Product` records, without historical snapshots.

These must be documented or adjusted before submission.

What each function does

Function	Purpose and limitation
positive()	Validates finite positive measurements; permits zero cost. Requires Decimal or integer inputs, avoiding float calculations.
dimensions()	Reads and validates the three dimensions. Uses the internal_ prefix for boxes.
volume()	Multiplies length, width, and height. It measures occupied volume, not packability.
orientations()	Returns sorted, unique permutations of the dimensions.
expand_items()	Creates one item per ordered unit, identified by order-item ID and unit number. Large quantities consume proportionally more memory.
overlaps()	Checks whether two items intersect along all three axes. Touching faces and edges are allowed.
pack_items()	Tries deterministic placements, returning a complete layout or None. Does not backtrack after a placement.
recommend_box()	Validates inputs, calculates totals, filters boxes, runs packing, and returns the first successful box in ranking order.

Rotations and quantities

For dimensions (L, W, H), the allowed orientations are:

(L, W, H) (L, H, W)

(W, L, H) (W, H, L)

(H, L, W) (H, W, L)

Duplicate orientations are removed. A cube therefore has only one distinct orientation. Diagonal tilting is excluded.

Quantity three means **three separate items**, each needing its own placement. Multiplying one dimension by quantity would incorrectly force a particular arrangement.

Boundary and overlap checks

Each placement records:

```
{  
  "order_item_id": 7,  
  "unit": 1,  
  "position": (x, y, z),  
  "size": (length, width, height),
```

}

The position is the item's lower corner. For every axis:

position ≥ 0

position + rotated dimension $\leq$ box dimension

Two items overlap only when their intervals intersect with positive length on **all three axes**. Separation along even one axis means they do not overlap.

Every new placement is checked against all previous placements.

Deterministic selection

The service fixes every relevant ordering:

- Boxes: cost, internal volume, then ID.
- Items: descending volume, descending longest side, then order-item ID and unit number.
- Positions: lowest z, then y, then x.
- Rotations: sorted dimension tuples.

This makes repeated calls with unchanged data produce the same result, regardless of the incoming box order.

Understanding success and failure

A successful result includes:

- status="recommended"
- Selected box ID and cost
- Total product weight and volume
- Every item's position and rotated dimensions
- Explanation and checks for boxes evaluated

Failure returns status="no_suitable_box" and per-box reasons:

Reason	Meaning
weight_exceeded	Impossible under the chosen weight policy
volume_exceeded	Impossible under the rectangular-item model
individual_item_too_large	Impossible with the allowed rotations
packing_not_found	This heuristic failed; another arrangement might work

Invalid input, including an empty order or nonpositive quantity, raises ValueError rather than being presented as a packing failure.

A complete placement proves geometric fit under the stated assumptions. Failure to find one does not prove impossibility.

Consequently, the service returns the **cheapest box for which it finds a valid packing**. It cannot guarantee the absolute cheapest feasible box, because it might miss a valid arrangement in a cheaper box.

Its main limitations are greedy placement, a restricted set of candidate positions, no backtracking, and no stability, cushioning, or load-bearing checks. With n expanded items, placement can take roughly **$O(n^3)$ per box**, making it appropriate for small orders rather than large warehouse optimization workloads.

I verified the earlier worked example, quantity totals, rotations, deterministic tie-breaking, rejection cases, and invalid inputs. Successful layouts were also independently checked for item count, allowed dimensions, boundaries, and pairwise non-overlap. Django integration remains to be checked against your actual models.

box_recommendation.pyCodeOpen file

```
Review the code below against our agreed requirements. Identify correctness bugs, unsupported
assumptions, validation gaps, unnecessary database queries, and places where the explanation
overstates what the packing algorithm guarantees. For each issue, explain a reproducible example
and a minimal fix. Do not rewrite the entire project.
import json from django.http import
JsonResponse from django.shortcuts import render from django.views.decorators.http import
require_POST from .models import Product, Box from .packing import select_box MAX_UNITS = 50
def home(request): return render(request, "shipping/home.html", {"products":
Product.objects.order_by("id"), "boxes": Box.objects.filter(active=True).order_by("cost", "id")})
@require_POST def recommend(request): if request.content_type !=
"application/json": return JsonResponse({"error": "Use application/json."},
status=415) try: data = json.loads(request.body) except (ValueError,
UnicodeDecodeError): return JsonResponse({"error": "Invalid JSON."}, status=400) if not
isinstance(data, dict) or set(data) != {"items":}: return JsonResponse({"error": "Provide an object
containing only items."}, status=400) items = data["items"] if not isinstance(items, list) or not 1
<= len(items) <= MAX_UNITS: return JsonResponse({"error": "Provide 1–50 item rows."},
status=400) quantities = {} for row in items: if not isinstance(row, dict) or set(row) !=
{"product_id", "quantity"} or any(type(row[k]) is not int or row[k] <= 0 for k in ("product_id",
"quantity")): return JsonResponse({"error": "Each row needs positive integer product_id and
quantity."}, status=400) quantities[row["product_id"]] = quantities.get(row["product_id"], 0) +
row["quantity"] if sum(quantities.values()) > MAX_UNITS: return JsonResponse({"error": "At
most 50 total units per request."}, status=400) if any(pk > 9223372036854775807 for pk in
quantities): return JsonResponse({"error": "Product ID out of range."}, status=400) products =
Product.objects.in_bulk(quantities) missing = sorted(set(quantities) - set(products)) if
missing: return JsonResponse({"error": "Unknown products.", "product_ids": missing},
status=400) units = [{"product_id": pk, "unit": n+1, "size": products[pk].dimensions} for pk, qty in
sorted(quantities.items()) for n in range(qty)] weight = sum(products[pk].weight_g * qty for pk, qty
in quantities.items()) return JsonResponse(select_box(units, weight,
Box.objects.filter(active=True)))
```

The request validation and quantity handling are mostly sound. **The main risks are the weight-unit contract, unsupported filtering and limits, and behaviour hidden inside select_box.** Without the models and packing function, several findings are conditional rather than confirmed bugs.

1. Weight units differ from our agreed contract

```
weight = sum(products[pk].weight_g * qty ...)
```

We agreed on kilograms; weight_g suggests grams.

Reproduce: Two products weighing 800 g each produce weight=1600. If select_box compares that with a capacity of 2 kg, it incorrectly rejects the box.

Minimal fix: Use weight_kg, or convert once using Decimal:

```
weight_kg = sum(  
    products[pk].weight_g * qty  
    for pk, qty in quantities.items()  
) / Decimal("1000")
```

Only do this if capacity is in kilograms. If both sides already use grams, packing may be correct, but the internal unit conversion needs documentation.

Similarly, verify that products[pk].dimensions returns **three positive dimensions in centimetres**, and that box dimensions are internal dimensions.

2. Cheapest-box selection and tie-breaking are not established here

```
Box.objects.filter(active=True)
```

The recommendation receives no explicit ordering. That is fine **if select_box sorts internally**.

Reproduce: Two fitting boxes cost ₹25. Box ID 1 has volume 8,000 cm³; box ID 2 has volume 4,000 cm³. Returning the first database row could incorrectly select ID 1.

Minimal fix: Sort inside select_box by:

```
(cost, internal_volume, id)
```

The home page's ordering by ("cost", "id") also omits volume, but this is only a display inconsistency unless that ordering drives selection.

3. active=True introduces an eligibility rule

Availability filtering was discussed as optional, not agreed as essential.

Reproduce: A fitting ₹10 box is inactive; a fitting ₹20 box is active. The service chooses ₹20, or reports no suitable box if none are active.

Minimal fix: Either remove the filter or explicitly document that recommendations consider **active boxes only**. Ensure the field exists; otherwise the query raises FieldError.

4. The 50-unit limit is a new restriction

A workload limit is reasonable for a heuristic, but 50 was not previously agreed.

Reproduce:

```
{"items": [{"product_id": 1, "quantity": 51}]}
```

This is rejected even if all 51 items fit.

Minimal fix: Keep it as an explicit, documented assignment limit, or revise it. Do not describe requests above the limit as packing failures.

The separate 50-row cap is logically redundant for valid data: every row contributes at least one unit. It can remain as an early rejection.

5. Stored measurements are trusted without visible validation

The view validates IDs and quantities, but not product dimensions, product weight, box capacity, or cost. These checks may already exist in `select_box` or database constraints.

Reproduce: With missing constraints, store a product with dimensions (0, 10, 10). A permissive algorithm might treat it as occupying zero volume. Negative weight could incorrectly reduce the total.

Minimal fix: Enforce positive measurements and capacity, and nonnegative cost, with database constraints. Validate numerical inputs once at the service boundary, including finiteness if non-database callers are supported.

Model validators alone are insufficient because Django does not automatically call `full_clean()` during `save()`. Django model validation documentation

6. Packaging and box-weight meanings remain implicit

Only product weight is summed, and dimensions pass through unchanged.

Reproduce:

- Products: 1.6 kg
- Internal packaging: 0.2 kg
- Empty box: 0.3 kg
- Gross shipment limit: 2 kg

The current product-only total passes, while gross weight is **2.1 kg**.

Likewise, an item exactly matching the box dimensions leaves no padding space.

Minimal fix: Explicitly retain the provisional policy—zero added clearance and product-only capacity—or implement a different agreed policy. Do not subtract empty-box weight unless the capacity actually means gross shipment weight.

7. Volume filtering and valid placement cannot be verified

The view does not calculate total volume. This is not a bug if `select_box` calculates it from the expanded units.

Reproduce: Two $2 \times 2 \times 2$ cubes in a $3 \times 3 \times 3$ box pass volume and individual-fit checks but cannot fit together.

Minimal fix: Check that `select_box`:

- Calculates volume across **every expanded unit**.
- Checks allowed rotations.

- Checks boundaries and pairwise non-overlap.
- Reports success only after placing every unit.

For quantity two, the result must contain two distinct placement records.

8. The response must not overstate the heuristic

No explanation text is visible here, so this depends on what `select_box` returns.

Reproduce: A cheaper box receives `packing_not_found` because of a greedy placement choice; a larger, more expensive box succeeds.

Calling the result “the cheapest possible box” would be unjustified. Calling the cheaper box “impossible to pack” would also be unjustified.

Minimal fix: Use wording such as:

Cheapest box for which this heuristic found a valid packing.

For failure:

No suitable box found. The heuristic may miss feasible arrangements.

Keep conclusive reasons such as `weight_exceeded` separate from `packing_not_found`. A geometric layout also does not establish stack stability or sufficient cushioning.

9. Failure and serialization depend on an undocumented return contract

```
return JsonResponse(select_box(...))
```

This requires `select_box` to return a JSON-serializable dictionary.

Reproduce: If no box fits and `select_box` returns `None`, the default `JsonResponse` rejects it. A raw `Box` instance is also unsuitable.

Minimal fix: Always return a consistent dictionary:

```
{
    "status": "no_suitable_box",
    "box_id": None,
    "explanation": "No valid packing found.",
    "checks": [],
}
```

A successful calculation finding no box can reasonably return HTTP 200. Invalid request data should remain a separate 400 result. Do not catch every exception and convert programming errors into “no suitable box.”

10. Some service responsibilities remain in the view

Quantity expansion and weight calculation happen here. This falls short of the earlier goal of a reusable service owning those operations.

Reproduce: A management command calling the packing service must duplicate this logic or construct units and totals itself, potentially supplying inconsistent weight.

Minimal fix: Move just the expansion and totals into a small service helper or wrapper. Keep request parsing and HTTP responses in the view; no project rewrite is needed.

Database-query review

There is no demonstrated N+1 query problem in this snippet.

Operation	Expected behaviour
Product.objects.in_bulk(quantities)	One product query for this small request
products[pk] inside comprehensions	Dictionary lookup; no additional query
Box queryset	Normally one query when evaluated
Home-page querysets	Normally two queries if both are rendered

Querysets are lazy, and ordinary repeated iteration over an evaluated queryset uses its cache. [Django QuerySet documentation](#)

Conditional query issues to inspect:

- **If dimensions queries a related record:** quantity 50 could cause repeated queries. Load that relation once and compute dimensions once per product.
- **If select_box calls .exists() and then iterates:** it may perform two box queries. For this small assignment, materialize the boxes once and check the resulting list.
- **If select_box queries boxes again internally:** reuse the supplied iterable.

Do not add select_related() without an actual related-object access to optimize.

What is already correct

- POST-only handling and JSON content-type validation.
- Rejecting malformed JSON and incorrect object shapes.
- Rejecting booleans, floats, strings, zero, and negative quantities through type(...) is int.
- Combining repeated product IDs before checking total units.
- Rejecting unknown products.
- Expanding quantities deterministically by product ID.
- Fetching products in bulk.

Two remaining design choices should simply be documented: repeated product rows are merged, and this endpoint calculates from current catalog data without saving an Order. Neither is automatically a correctness bug.

Help me write a README covering setup, assumptions, data models, recommendation logic, one worked example, test commands and known limitations. Also help me prepare AI_USAGE.md containing only: - Tools used. - Prompts actually used. - Outputs I accepted or rejected and why. - Mistakes found. - Verification steps and actual results. Use the development notes I provide. Do not invent prompts, independent work, rejected suggestions, or successful test results. Ask for missing facts.

Please paste your development notes and current test output. I can use the prompts visible in this conversation, but I need these facts to make both files accurate:

1. **Setup:** Python/Django versions, dependency filename, project/app names, database, and commands for loading sample data.
2. **Final implementation:** Actual model fields and packing algorithm. Which policies did you keep for packaging clearance, weight units, box capacity, active boxes, and the 50-unit limit?
3. **Worked example:** Sample products, quantities, available boxes, and the actual recommendation returned.
4. **Tests:** Exact commands and their terminal output, including failures or warnings.
5. **AI usage:** Tools you actually used, suggestions or code you incorporated, anything you rejected or modified and why, and mistakes you confirmed.

You can paste rough notes in this format:

Setup:

Final models and assumptions:

Packing implementation:

Sample input and actual result:

Test commands and output:

AI tools used:

Outputs accepted:

Outputs rejected or modified, and why:

Confirmed mistakes:

I'll draft README.md and AI_USAGE.md from that evidence. I'll distinguish tests I ran on the standalone service from tests you ran in your Django project, and won't treat a suggestion as accepted merely because it appeared in our chat.

Python/Django Hiring Assignment: AI-Assisted Box Selection System

Context

We operate an ecommerce platform. When a customer places an order, the warehouse team needs to know which shipping box should be used. Each product has dimensions and weight. Each box has internal dimensions, maximum weight capacity, and cost.

Your task is to design and build a small Django-based system that recommends the most suitable box for an order.

You are allowed to use AI tools such as ChatGPT, Claude, Cursor, GitHub Copilot, or similar tools. However, your submission must clearly show your own thinking.

Download `Box_Selection_Django_Assignment.zip`

Built a runnable project containing:

- Django models and admin for products and boxes.
- Order-entry page and recommendation API.
- Weight checks, 3D packing, rotations, and cost-based selection.
- **14 passing tests.**
- Windows setup instructions in `README.md`.
- Design explanations and an AI-use disclosure worksheet.

The packing heuristic can miss some valid arrangements; this limitation is documented. Before submitting, review the code and complete `AI_USAGE.md` with your actual decisions and changes to demonstrate your own thinking.

AI Usage

You may use AI tools, but include an `AI_USAGE.md` file containing:

1. Which AI tool(s) you used.
2. The prompts you gave.
3. What output you accepted.
4. What output you rejected or modified.
5. Any mistakes the AI made.
6. How you verified the final code.

Submission

Please
submit:

1.
GitHub repository link.
2.
README.md.
3.
AI_USAGE.md describing tools, prompts, accepted/rejected outputs, mistakes found, and verification steps.
4.
Export the chat transcript and attach and include in your repository. Make sure that you don't generate this using AI. Your assignment will be rejected immediately if we see that you used AI for this.
5.
What did you learn in this assignment? Again, not to use AI to generate an answer for this.
6.
Test cases.
7.
Test run output, either:

-

GitHub Actions link,

- terminal output pasted in TEST_OUTPUT.md. there is also these condions

Yes—these conditions add important submission requirements. I've updated Box_Selection_Django_Assignment.zip with:

- AI_USAGE.md covering all six points honestly.
- TEST_OUTPUT.md containing actual terminal output—**14 tests passed**.
- Updated README and submission checklist.

Still required before submitting:

1. Upload the extracted project to GitHub and provide the repository link.
2. Export the actual chat directly, attach it, and include it in the repository. An AI-generated transcript is not acceptable.
3. Write **"What did you learn?" yourself**, without AI.
4. Review the code and update AI_USAGE.md with your actual accepted outputs, changes, and verification.

The package does not include a generated transcript or learning answer.

Anuj, is output mein **14 tests pass hain**, lekin ye **assistant ke Linux environment ka run** hai. Tumhare Windows project ka actual output capture karne ke liye project folder mein ye command run karo:

```
.venv\Scripts\python.exe manage.py test --verbosity 2 > test_output.txt 2>&1
```

Isse complete test output **test_output.txt** mein save ho jayega.

Python aur Django versions check karo:

```
.venv\Scripts\python.exe --version
```

```
.venv\Scripts\python.exe -m django --version
```

Phir TEST_OUTPUT.md mein:

- Apne actual Python aur Django versions likho.
- Platform Windows likho.
- Actual test-run date likho.
- test_output.txt ka content **as-is** fenced text code block mein paste karo.

Sirf existing Linux output ka platform Windows mat change karna, kyunki woh alag environment ka evidence hai.

Command run karke test_output.txt ka output bhejo; main usko proper Markdown format mein arrange kar dunga.

```
Creating test database for alias 'default' ('file:memorydb_default?mode=memory&cache=shared')...
Found 14 test(s). Operations to perform: Synchronize unmigrated apps: messages, staticfiles Apply all
migrations: admin, auth, contenttypes, sessions, shipping Synchronizing apps without migrations:
Creating tables... Running deferred SQL... Running migrations: Applying contenttypes.0001_initial...
OK Applying auth.0001_initial... OK Applying admin.0001_initial... OK Applying
admin.0002_logentry_remove_auto_add... OK Applying
admin.0003_logentry_add_action_flag_choices... OK Applying
contenttypes.0002_remove_content_type_name... OK Applying
auth.0002_alter_permission_name_max_length... OK Applying
auth.0003_alter_user_email_max_length... OK Applying auth.0004_alter_user_username_opts... OK
Applying auth.0005_alter_user_last_login_null... OK Applying
auth.0006_require_contenttypes_0002... OK Applying
auth.0007_alter_validators_add_error_messages... OK Applying
auth.0008_alter_user_username_max_length... OK Applying
auth.0009_alter_user_last_name_max_length... OK Applying
auth.0010_alter_group_name_max_length... OK Applying auth.0011_update_proxy_permissions...
OK Applying auth.0012_alter_user_first_name_max_length... OK Applying sessions.0001_initial... OK
Applying shipping.0001_initial...test_cost_and_volume_tie
(shipping.tests.APITests.test_cost_and_volume_tie) ... ok test_db_constraint
(shipping.tests.APITests.test_db_constraint) ... ok test_dimension_and_volume_rejection
(shipping.tests.APITests.test_dimension_and_volume_rejection) ... ok test_duplicates
(shipping.tests.APITests.test_duplicates) ... ok test_http_errors
(shipping.tests.APITests.test_http_errors) ... ok test_inactive (shipping.tests.APITests.test_inactive) ...
ok test_invalid (shipping.tests.APITests.test_invalid) ... ok test_page_and_csrf
(shipping.tests.APITests.test_page_and_csrf) ... ok test_unknown
```

```
(shipping.tests.APITests.test_unknown) ... ok test_weight_boundary
(shipping.tests.APITests.test_weight_boundary) ... ok test_exact_fit
(shipping.tests.PackingTests.test_exact_fit) ... ok test_randomized_geometry
(shipping.tests.PackingTests.test_randomized_geometry) ... ok test_rotation
(shipping.tests.PackingTests.test_rotation) ... ok test_volume_is_not_enough
(shipping.tests.PackingTests.test_volume_is_not_enough) ... ok -----
----- Ran 14 tests in 0.058s OK Destroying test database for alias 'default'
('file:memorydb_default?mode=memory&cache=shared')... OK System check identified no issues (0
silenced).
```

Tumhare shared output mein sabhi 14 tests pass hue hain—0.058 seconds mein. Django system checks mein bhi koi issue nahi mila.

Neeche ka content **TEST_OUTPUT.md** mein paste kar do. Python/Django versions aur run date output mein nahi hain, isliye unhe assume nahi kiya hai.

Test Run Output

Test Run Output

Local test output supplied by the candidate.

Command

```
.venv\Scripts\python.exe manage.py test --verbosity 2 > test_output.txt 2>&1
```

Result

- Tests discovered: 14
- Tests passed: 14
- Execution time: 0.058 seconds
- Final result: OK
- Django system checks: No issues identified

Captured Terminal Output

Creating test database for alias 'default' ('file:memorydb_default?mode=memory&cache=shared')...

Found 14 test(s).

Operations to perform:

Synchronize unmigrated apps: messages, staticfiles

Apply all migrations: admin, auth, contenttypes, sessions, shipping

Synchronizing apps without migrations:

Creating tables...Export the chat transcript and attach and include in your repository.

Running deferred SQL...

Running migrations:

Applying contenttypes.0001_initial... OK

Applying auth.0001_initial... OK

Applying admin.0001_initial... OK

Applying admin.0002_logentry_remove_auto_add... OK

Applying admin.0003_logentry_add_action_flag_choices... OK

Applying contenttypes.0002_remove_content_type_name... OK

Applying auth.0002_alter_permission_name_max_length... OK

Applying auth.0003_alter_user_email_max_length... OK

Applying auth.0004_alter_user_username_opts... OK

Applying auth.0005_alter_user_last_login_null... OK

Applying auth.0006_require_contenttypes_0002... OK

Applying auth.0007_alter_validators_add_error_messages... OK

Applying auth.0008_alter_user_username_max_length... OK

Applying auth.0009_alter_user_last_name_max_length... OK

Applying auth.0010_alter_group_name_max_length... OK

Applying auth.0011_update_proxy_permissions... OK

Applying auth.0012_alter_user_first_name_max_length... OK

Applying sessions.0001_initial... OK

Applying shipping.0001_initial...test_cost_and_volume_tie
(shipping.tests.APITests.test_cost_and_volume_tie) ... ok

test_db_constraint (shipping.tests.APITests.test_db_constraint) ... ok

test_dimension_and_volume_rejection
(shipping.tests.APITests.test_dimension_and_volume_rejection) ... ok

test_duplicates (shipping.tests.APITests.test_duplicates) ... ok

test_http_errors (shipping.tests.APITests.test_http_errors) ... ok

test_inactive (shipping.tests.APITests.test_inactive) ... ok

test_invalid (shipping.tests.APITests.test_invalid) ... ok

test_page_and_csrf (shipping.tests.APITests.test_page_and_csrf) ... ok

test_unknown (shipping.tests.APITests.test_unknown) ... ok

test_weight_boundary (shipping.tests.APITests.test_weight_boundary) ... ok

test_exact_fit (shipping.tests.PackingTests.test_exact_fit) ... ok

test_randomized_geometry (shipping.tests.PackingTests.test_randomized_geometry) ... ok
test_rotation (shipping.tests.PackingTests.test_rotation) ... ok
test_volume_is_not_enough (shipping.tests.PackingTests.test_volume_is_not_enough) ... ok

Ran 14 tests in 0.058s

OK

Destroying test database for alias 'default'
('file:memorydb_default?mode=memory&cache=shared')...

OK

System check identified no issues (0 silenced).

Ab AI_USAGE.md mein “**local test run pending**” ko update kar sakte ho: local suite run ho chuki hai aur uska output share kiya gaya hai.

Export the chat transcript and attach and include in your repository. explain me what is this?

Anuj, iska matlab hai: **project banate waqt tumne ChatGPT se jo actual conversation ki hai—tumhare prompts aur ChatGPT ke replies—uski original copy submit karni hai.**

Unhe ye 3 cheezein chahiye:

1. **Export the chat transcript:** Actual assignment conversation ko directly export/save karo. ChatGPT se transcript likhwaana ya reconstruct karwana allowed nahi hai.
2. **Attach it:** Assignment submit karte waqt exported file ko email ya submission form mein attach karo.
3. **Include in your repository:** Wahi file GitHub project mein bhi upload karo, jaise docs/chat-transcript.pdf—agar PDF format employer accept karta ho.

AI_USAGE.md aur transcript alag hain:

File	Kya hota hai
------	--------------

AI_USAGE.md	AI tools, prompts, changes aur verification ka summary
-------------	--

Chat transcript	Actual prompts aur replies ka original conversation record
-----------------	--

Agar project ke liye multiple chats use ki hain, unke records bhi include karo. Sirf selected “achhe prompts” ki list transcript nahi hoti.

AI usage disclosure

This document describes the AI-assisted work visible in this conversation.

It was drafted by ChatGPT. It is not a chat transcript or a candidate-authored learning reflection. Candidate acceptance and independent verification have not yet been recorded; the candidate must review and update those facts before submission.

1. AI tools used

ChatGPT in Work mode generated and edited the initial Django project, documentation, and tests, and invoked a Python/shell environment to execute commands. Django documentation was consulted through web search. No other AI coding tools were used in the visible work. The application itself does not call an LLM or require an AI API key.

2. Prompts supplied

The following records the assignment prompts supplied by the user; whitespace and Markdown formatting are normalized. It does not recreate the conversation or replace the required original chat export.

Prompt 1: initial assignment

> Python/Django Hiring Assignment: AI-Assisted Box Selection System

>

> Context

>

> We operate an ecommerce platform. When a customer places an order, the warehouse team needs to know which shipping box should be used. Each product has dimensions and weight. Each box has internal dimensions, maximum weight capacity, and cost.

>

> Your task is to design and build a small Django-based system that recommends the most suitable box for an order.

>

> You are allowed to use AI tools such as ChatGPT, Claude, Cursor, GitHub Copilot, or similar tools. However, your submission must clearly show your own thinking.

Prompt 2: additional conditions

> AI Usage

>

> You may use AI tools, but include an AI_USAGE.md file containing:

>

> 1. Which AI tool(s) you used.

> 2. The prompts you gave.

> 3. What output you accepted.

> 4. What output you rejected or modified.

> 5. Any mistakes the AI made.

> 6. How you verified the final code.

>

> Submission

>

> Please submit:

>

> 1. GitHub repository link.

> 2. README.md.

> 3. AI_USAGE.md describing tools, prompts, accepted/rejected outputs, mistakes found, and verification steps.

> 4. Export the chat transcript and attach and include in your repository. Make sure that you don't generate this using AI. Your assignment will be rejected immediately if we see that you used AI for this.

> 5. What did you learn in this assignment? Again, not to use AI to generate an answer for this.

> 6. Test cases.

> 7. Test run output, either:

> - GitHub Actions link,

> - terminal output pasted in TEST_OUTPUT.md.

>

> there is also these conditions

Any later prompts and any use of additional AI tools must be added by the candidate. No separate user prompt specifically choosing the packing heuristic, ranking policy, or database was supplied; those were AI-proposed assumptions.

3. Output retained in this project

The generated baseline retains the following AI outputs:

- Django Product and Box models, validation constraints, and migrations.
- Admin registration, demo catalog command, order form, and JSON API.
- Deterministic 3D packing heuristic, weight/volume/dimension filters, and cost-first selection with placement coordinates and limitation disclosures.
- Fourteen automated tests, setup instructions, and design documentation.
- This expanded disclosure, submission checklist, and a test-output document whose terminal output is captured directly from subprocess execution.

These are outputs retained by the assistant in the delivered files. This does not claim that the candidate has independently reviewed or accepted them.

Candidate acceptance: not yet recorded.

4. Output rejected or modified

No candidate rejection or candidate code modification has been recorded yet.

The assistant made these revisions:

- Corrected a failed test-file write caused by using the wrong relative path.
- Replaced the original general AI-use worksheet with this six-part record after receiving the additional submission conditions.
- Added TEST_OUTPUT.md with real verbose test output and a submission checklist.

The application source was not changed during the documentation revision.

The heuristic may miss feasible packings; it is not presented as an exact optimizer. No fabricated transcript or AI-written learning answer is included.

5. Mistakes and gaps found

1. ****Wrong test-file path:**** while already inside `box-selection`, the assistant attempted to write `box-selection/shipping/tests.py`. The shell reported `No such file or directory`. It was corrected to `shipping/tests.py`, and Django then discovered and passed 14 tests.
2. ****Initial documentation gap:**** the first package's AI_USAGE.md did not enumerate prompts, accepted/rejected output, or the observed mistake. It also lacked TEST_OUTPUT.md. These requirements were supplied in Prompt 2 and are addressed in this revision.
3. ****Unconfirmed design assumptions:**** cost-first ranking, single-box orders,

unrestricted cuboid rotations, and no runtime LLM were chosen without a detailed rubric. These remain assumptions in DESIGN.md, not confirmed employer requirements. The candidate should check them against the full brief.

No packing failure was observed in the executed tests. Passing tests do not establish the absence of bugs or completeness of the heuristic search.

6. Verification performed

The assistant's execution environment, not the candidate's local machine, performed these checks:

- `python manage.py makemigrations shipping`: generated the initial migration.
- `python manage.py migrate`: applied migrations successfully to SQLite.
- `python manage.py seed\_demo`: created the sample catalog successfully.
- `python manage.py check`: no system-check issues.
- `python manage.py test`: 14 tests passed in the initial run.
- A Django test-client request for two seeded Books: Small box, cost 15.00, total weight 800 g, utilization 59.66%, matching the README example.
- `python manage.py test --verbosity 2`: rerun to capture detailed actual output in TEST_OUTPUT.md, including test names, environment, and exit status.

Tests cover rotations, exact fit, a volume-only counterexample, 100 seeded randomized geometric cases, cost ranking, weight limits, inactive boxes, unknown products, invalid input, duplicate quantities, HTTP handling, CSRF, database constraints, and dimension/volume rejection. Randomized checks verify successful placements for containment, non-overlap, and item preservation.

No real-browser automation, production deployment test, concurrency/load test, or candidate-performed local test run is claimed. Candidate verification:

not yet recorded. After reviewing or changing the project, the candidate should

run the suite and retain the output for the exact code submitted. # AI_USAGE.md describing tools, prompts, accepted/rejected outputs, mistakes found, and verification steps.

Neeche updated content **AI_USAGE.md** mein paste kar sakte ho. Isme tumhara successful seed_demo run aur documentation follow-ups bhi included hain. Personal code review aur final local tests ka status apne actual work ke according update karna.

AI Usage Disclosure

AI Usage Disclosure

This document records AI assistance used during the development and documentation of the Box Selection System. It was drafted with ChatGPT assistance.

It is not the original chat transcript or the candidate's personal learning response. The genuine conversation export must be submitted separately.

1. AI Tools Used

ChatGPT in Work mode was used to:

- Generate the initial Django project.
- Propose the box-selection policy and packing algorithm.
- Create models, views, templates, migrations, and automated tests.
- Execute setup commands and tests in the assistant's execution environment.
- Explain local setup and application behavior.
- Draft and revise project documentation.

The assistant also consulted Django documentation through web search.

No other AI coding tool is recorded in this conversation. Any additional tools used outside this conversation must be disclosed separately.

The application itself does not call an LLM or require an AI API key.

2. Prompts Supplied

The following table records the requests made during this conversation. Long pasted documents are described rather than reproduced in full; short excerpts preserve the user's wording.

This prompt record does not replace the required original chat export.

Request	Prompt or Description
Initial implementation	Supplied the "Python/Django Hiring Assignment: AI-Assisted Box Selection System" brief, describing products, box dimensions, weight capacity, cost, and the need to recommend a suitable box.
Submission requirements	Supplied the six AI-disclosure requirements and the seven submission requirements, including the original chat export, personally written learning response, test cases, and test evidence.
Implementation guidance	"explain me how to impliment step by step this project"
Local setup progress	Shared the command <code>.venv\Scripts\python.exe manage.py seed_demo</code> and its output: Demo catalog ready.
Weight and utilization explanation	"in large box the quntity is 12 but space utilization is still not used full capacity but box cannot take more then 12 books why?"

Design review	Pasted the design document and asked: “this is correct?”
Design revision	“change karo accordingly so i put it”
README revision	Pasted the README and asked: “isko correct aur suitable and understandable formate me rakhe”
Submission checklist revision	Pasted the checklist and asked: “isko bhi correct kijye”
AI disclosure revision	Pasted the existing disclosure and requested AI_USAGE.md covering tools, prompts, accepted/rejected outputs, mistakes, and verification.

The packing heuristic, cost-first ranking, SQLite database, and single-box scope were proposed by the assistant rather than explicitly selected in the initial prompt.

Any further prompts or assignment-related conversations must also be disclosed, and their genuine exports included as required.

3. Outputs Retained and Used

The delivered project contains these AI-generated outputs:

- Django Product and Box models with validation constraints.
- Initial database migration.
- Django admin registration.
- Sample catalog command.
- Browser order-entry form.
- JSON recommendation endpoint.
- Weight, volume, and individual-dimension checks.
- A deterministic 3D packing heuristic with product rotations.
- Cost-based box selection and placement details.
- Fourteen automated tests.
- Setup instructions and design documentation.
- AI-use disclosure, submission checklist, and captured test output.

The candidate shared a successful local execution of the generated `seed_demo` command and asked about the application’s handling of 12 Books in the Large box.

The candidate also requested revisions to the design document, README, checklist, and AI disclosure.

These observations show use of the project and documentation assistance. They do not establish that every generated source file has been independently reviewed or accepted.

Candidate’s final acceptance record: To be completed after reviewing the submitted files.

4. Outputs Rejected or Modified

No specific candidate-authored source-code modification or explicit rejection of generated code has been recorded in this conversation.

The following assistant revisions were made:

Revision	Reason
Corrected the test-file write location	The initial command used an incorrect relative path.
Expanded the initial AI-use worksheet	The additional requirements requested prompts, retained/rejected outputs, mistakes, and verification details.
Added TEST_OUTPUT.md	The assignment required actual test-run evidence.
Clarified the design document	Explained duplicate packing attempts, temporary candidate-list memory, and separately validated randomized test results.
Added the weight-versus-volume example	Explained why unused space does not permit exceeding a box's weight capacity.
Reformatted and expanded the README	Made setup, API usage, units, testing, and limitations easier to follow.
Revised the submission checklist	Distinguished prepared files from outstanding personal submission tasks.
Updated this disclosure	Included subsequent prompts and the candidate's observed local setup progress.

The later documentation revisions did not change application source code.

No AI-generated chat transcript or personal learning response was produced.

Candidate's additional modifications or rejections: To be completed if applicable.

5. Mistakes and Gaps Found

Incorrect Test-File Path

While already inside the box-selection directory, the assistant attempted to write:

box-selection/shipping/tests.py

The shell returned:

No such file or directory

The assistant corrected the destination to:

shipping/tests.py

Django then discovered and passed 14 tests.

Initial Disclosure Was Incomplete for the Additional Requirements

The first package contained a general AI-use worksheet but did not provide the complete six-part disclosure or a separate test-output file.

After the additional submission conditions were supplied, the assistant expanded AI_USAGE.md and added TEST_OUTPUT.md.

Ambiguous Randomized-Test Wording

The original phrase “independent randomized test” could imply that someone independently authored the tests.

The wording was revised to explain that an AI-generated, seeded randomized test separately validates successful placements for containment, non-overlap, and preservation of product dimensions.

Unconfirmed Design Assumptions

The assistant selected these assumptions without explicit confirmation in the supplied brief:

- Lowest box purchase cost takes priority.
- Each order uses one box.
- Products are represented as rigid cuboids.
- Axis-aligned rotations are permitted.
- Padding and physical stacking constraints are excluded.
- A runtime LLM is not required.

These are documented assumptions, not confirmed employer requirements.

Packing Limitation

The algorithm is a heuristic and may miss feasible arrangements. It therefore cannot guarantee the globally cheapest feasible box.

This is a known design limitation, not an observed test failure. All recorded automated tests passed, including tests that intentionally expect packing rejection.

6. Verification Performed

Assistant-Environment Verification

The assistant executed the following checks in its own environment:

Command or Check	Observed Result
python manage.py makemigrations shipping	Generated the initial migration.
python manage.py migrate	Applied migrations successfully to SQLite.
python manage.py seed_demo	Created the sample catalog successfully.
python manage.py check	Reported no system-check issues.

<code>python manage.py test</code>	Passed 14 tests.
<code>python manage.py test --verbosity 2</code>	Passed 14 tests and captured actual output in TEST_OUTPUT.md.
Django test-client request for two sample Books	Returned Small box, cost 15.00, total weight 800 g, and utilization 59.66%.

The recorded assistant environment used Django 5.2.17.

Automated Test Coverage

The tests cover:

- Product rotation.
- Exact-fit packing.
- A volume-only counterexample.
- Weight limits.
- Box-cost ranking and volume tie-breaking.
- Inactive boxes.
- Unknown products.
- Invalid request data.
- Repeated product quantities.
- HTTP handling.
- CSRF protection.
- Database constraints.
- Dimension and volume rejection.

One test generates 100 seeded randomized cases. When packing succeeds, it checks that:

- All expected units are present.
- Product dimensions are preserved under rotation.
- Products remain inside the box.
- Products do not overlap.

These tests verify the checked cases, not completeness of the heuristic.

Candidate-Reported Local Verification

The candidate shared this Windows command:

```
.venv\Scripts\python.exe manage.py seed_demo
```

Observed output:

Demo catalog ready.

The candidate also reported that the Large box accepted 12 Books but not more, despite unused space.

The assistant explained this using the original sample values:

- Each Book weighs 400 g.
- The Large box supports 5,000 g.
- 12 Books weigh 4,800 g.
- 13 Books weigh 5,200 g and exceed capacity.

This records the candidate's reported application observation. It is not a substitute for a captured local test-suite run.

Verification Still to Record

A candidate-performed full local test run has not been shared in this conversation.

Before submission, run:

```
.venv\Scripts\python.exe manage.py test --verbosity 2
```

```
.venv\Scripts\python.exe manage.py check
```

Record the actual output for the final submitted code in TEST_OUTPUT.md, or provide a corresponding GitHub Actions run link.

No automated real-browser test, production deployment test, or concurrency/load test is claimed.

Anuj, is output mein **14 tests pass hain**, lekin ye **assistant ke Linux environment ka run** hai. Tumhare Windows project ka actual output capture karne ke liye project folder mein ye command run karo:

```
.venv\Scripts\python.exe manage.py test --verbosity 2 > test_output.txt 2>&1
```

Isse complete test output **test_output.txt** mein save ho jayega.

Python aur Django versions check karo:

```
.venv\Scripts\python.exe --version
```

```
.venv\Scripts\python.exe -m django --version
```

Phir TEST_OUTPUT.md mein:

- Apne actual Python aur Django versions likho.
- Platform Windows likho.
- Actual test-run date likho.
- test_output.txt ka content **as-is** fenced text code block mein paste karo.

Sirf existing Linux output ka platform Windows mat change karna, kyunki woh alag environment ka evidence hai.

Command run karke test_output.txt ka output bhejo; main usko proper Markdown format mein arrange kar dunga.

```
Creating test database for alias 'default' ('file:memorydb_default?mode=memory&cache=shared')...
Found 14 test(s). Operations to perform: Synchronize unmigrated apps: messages, staticfiles Apply all
migrations: admin, auth, contenttypes, sessions, shipping Synchronizing apps without migrations:
Creating tables... Running deferred SQL... Running migrations: Applying contenttypes.0001_initial...
OK Applying auth.0001_initial... OK Applying admin.0001_initial... OK Applying
admin.0002_logentry_remove_auto_add... OK Applying
admin.0003_logentry_add_action_flag_choices... OK Applying
contenttypes.0002_remove_content_type_name... OK Applying
auth.0002_alter_permission_name_max_length... OK Applying
auth.0003_alter_user_email_max_length... OK Applying auth.0004_alter_user_username_opts... OK
Applying auth.0005_alter_user_last_login_null... OK Applying
auth.0006_require_contenttypes_0002... OK Applying
auth.0007_alter_validators_add_error_messages... OK Applying
auth.0008_alter_user_username_max_length... OK Applying
auth.0009_alter_user_last_name_max_length... OK Applying
auth.0010_alter_group_name_max_length... OK Applying auth.0011_update_proxy_permissions...
OK Applying auth.0012_alter_user_first_name_max_length... OK Applying sessions.0001_initial... OK
Applying shipping.0001_initial...test_cost_and_volume_tie
(shipping.tests.APITests.test_cost_and_volume_tie) ... ok test_db_constraint
(shipping.tests.APITests.test_db_constraint) ... ok test_dimension_and_volume_rejection
(shipping.tests.APITests.test_dimension_and_volume_rejection) ... ok test_duplicates
(shipping.tests.APITests.test_duplicates) ... ok test_http_errors
(shipping.tests.APITests.test_http_errors) ... ok test_inactive (shipping.tests.APITests.test_inactive) ...
ok test_invalid (shipping.tests.APITests.test_invalid) ... ok test_page_and_csrf
(shipping.tests.APITests.test_page_and_csrf) ... ok test_unknown
(shipping.tests.APITests.test_unknown) ... ok test_weight_boundary
(shipping.tests.APITests.test_weight_boundary) ... ok test_exact_fit
(shipping.tests.PackingTests.test_exact_fit) ... ok test_randomized_geometry
(shipping.tests.PackingTests.test_randomized_geometry) ... ok test_rotation
(shipping.tests.PackingTests.test_rotation) ... ok test_volume_is_not_enough
(shipping.tests.PackingTests.test_volume_is_not_enough) ... ok -----
----- Ran 14 tests in 0.058s OK Destroying test database for alias 'default'
('file:memorydb_default?mode=memory&cache=shared')... OK System check identified no issues (0
silenced).
```

Tumhare shared output mein sabhi 14 tests pass hue hain—0.058 seconds mein. Django system checks mein bhi koi issue nahi mila.

Neeche ka content **TEST_OUTPUT.md** mein paste kar do. Python/Django versions aur run date output mein nahi hain, isliye unhe assume nahi kiya hai.

Test Run Output

Test Run Output

Local test output supplied by the candidate.

Command

```
.venv\Scripts\python.exe manage.py test --verbosity 2 > test_output.txt 2>&1
```

Result

- Tests discovered: 14
- Tests passed: 14
- Execution time: 0.058 seconds
- Final result: OK
- Django system checks: No issues identified

Captured Terminal Output

Creating test database for alias 'default' ('file:memorydb_default?mode=memory&cache=shared')...

Found 14 test(s).

Operations to perform:

Synchronize unmigrated apps: messages, staticfiles

Apply all migrations: admin, auth, contenttypes, sessions, shipping

Synchronizing apps without migrations:

Creating tables...Export the chat transcript and attach and include in your repository.

Running deferred SQL...

Running migrations:

Applying contenttypes.0001_initial... OK

Applying auth.0001_initial... OK

Applying admin.0001_initial... OK

Applying admin.0002_logentry_remove_auto_add... OK

Applying admin.0003_logentry_add_action_flag_choices... OK

Applying contenttypes.0002_remove_content_type_name... OK

Applying auth.0002_alter_permission_name_max_length... OK

Applying auth.0003_alter_user_email_max_length... OK

Applying auth.0004_alter_user_username_opts... OK

Applying auth.0005_alter_user_last_login_null... OK

Applying auth.0006_require_contenttypes_0002... OK

Applying auth.0007_alter_validators_add_error_messages... OK

Applying auth.0008_alter_user_username_max_length... OK

Applying auth.0009_alter_user_last_name_max_length... OK
Applying auth.0010_alter_group_name_max_length... OK
Applying auth.0011_update_proxy_permissions... OK
Applying auth.0012_alter_user_first_name_max_length... OK
Applying sessions.0001_initial... OK
Applying shipping.0001_initial...test_cost_and_volume_tie
(shipping.tests.APITests.test_cost_and_volume_tie) ... ok
test_db_constraint (shipping.tests.APITests.test_db_constraint) ... ok
test_dimension_and_volume_rejection
(shipping.tests.APITests.test_dimension_and_volume_rejection) ... ok
test_duplicates (shipping.tests.APITests.test_duplicates) ... ok
test_http_errors (shipping.tests.APITests.test_http_errors) ... ok
test_inactive (shipping.tests.APITests.test_inactive) ... ok
test_invalid (shipping.tests.APITests.test_invalid) ... ok
test_page_and_csrf (shipping.tests.APITests.test_page_and_csrf) ... ok
test_unknown (shipping.tests.APITests.test_unknown) ... ok
test_weight_boundary (shipping.tests.APITests.test_weight_boundary) ... ok
test_exact_fit (shipping.tests.PackingTests.test_exact_fit) ... ok
test_randomized_geometry (shipping.tests.PackingTests.test_randomized_geometry) ... ok
test_rotation (shipping.tests.PackingTests.test_rotation) ... ok
test_volume_is_not_enough (shipping.tests.PackingTests.test_volume_is_not_enough) ... ok

Ran 14 tests in 0.058s

OK

Destroying test database for alias 'default'
('file:memorydb_default?mode=memory&cache=shared')...

OK

System check identified no issues (0 silenced).

Ab AI_USAGE.md mein **“local test run pending”** ko update kar sakte ho: local suite run ho chuki hai aur uska output share kiya gaya hai.

Export the chat transcript and attach and include in your repository. explain me what is this?

Anuj, iska matlab hai: **project banate waqt tumne ChatGPT se jo actual conversation ki hai—tumhare prompts aur ChatGPT ke replies—uski original copy submit karni hai.**

Unhe ye 3 cheezein chahiye:

1. **Export the chat transcript:** Actual assignment conversation ko directly export/save karo. ChatGPT se transcript likhwaana ya reconstruct karwana allowed nahi hai.
2. **Attach it:** Assignment submit karte waqt exported file ko email ya submission form mein attach karo.
3. **Include in your repository:** Wahi file GitHub project mein bhi upload karo, jaise docs/chat-transcript.pdf—agar PDF format employer accept karta ho.